第七章 卷积神经网络

本章的核心目标是：

- 理解为什么全连接网络不适合直接处理图像
- 掌握卷积层与池化层的基本思想
- 理解特征图、通道、滤波器等核心概念
- 学会卷积层输出大小的计算方法
- 明白 `im2col` 的作用
- 了解 CNN 的整体结构
- 对经典卷积网络有初步认识

7.1 整体结构

在前面的章节中，我们主要讨论的是全连接神经网络。

全连接层的特点是：

- 当前层的每个神经元都和前一层所有神经元相连
- 可以处理一般的向量输入
- 结构比较统一

但是如果直接用全连接网络处理图像，就会遇到一个很大的问题：

> 图像原本的空间结构会被破坏。

例如，一张大小为 28×28 的灰度图像，本来是一个二维结构。如果直接送入全连接网络，通常要先拉平成一个长度为 784 的一维向量。

这样做虽然能输入网络，但会丢失很多重要信息，例如：

- 像素之间的相邻关系
- 局部区域的结构
- 边缘、纹理等空间模式

因此，对于图像任务，更自然的做法是：

> 保留图像的二维甚至三维结构，让网络直接在局部区域上提取特征。

这就是卷积神经网络，也就是 CNN 出现的原因。

7.2 全连接层的问题

为了更直观地理解 CNN 的动机，先回顾一下全连接层的局限。

如果输入是一张图像，那么图像通常具有以下特点：

- 近邻像素之间高度相关
- 某些局部模式会反复出现
- 图像的平移不会改变语义本质

但全连接层对这些结构并不敏感：

- 它把所有输入位置都平等对待
- 它不知道哪些像素是相邻的
- 它不能天然利用局部模式的重复性

所以对于图像而言，全连接层存在两个主要问题：

7.2.1 参数量过大

如果输入图像分辨率稍大，参数数量会迅速膨胀。

7.2.2 空间结构被破坏

图像中的二维排列信息在展平后消失了。
因此我们需要一种新的层，它能够：

- 保留空间结构
- 利用局部连接
- 共享参数

这就是卷积层。

7.3 卷积层

卷积层最核心的思想是：

> 用一个小窗口在输入图像上滑动，并在每个位置做局部计算。

这个小窗口通常叫做：

- 卷积核
- 滤波器
- filter

例如，一个 3×3 的滤波器只会看输入中的一个 3×3 区域，然后输出一个数值。
接着它继续滑动到下一个位置，再重复同样的计算。
这样做的好处是：

- 只关注局部区域
- 参数数量大幅减少
- 可以捕捉局部模式，例如边缘和纹理

7.3.1 卷积运算的直观理解

假设输入是一个二维矩阵，滤波器也是一个二维矩阵。

卷积操作本质上就是：

1. 取输入中的一个局部区域
2. 与滤波器对应元素相乘
3. 全部加起来得到一个输出值

例如：

$$\text{输入局部区域} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \qquad \text{滤波器} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

那么输出值就是：

$$1 \cdot 1 + 2 \cdot 0 + 3 \cdot 1 + 4 \cdot 0 + 5 \cdot 1 + 6 \cdot 0 + 7 \cdot 1 + 8 \cdot 0 + 9 \cdot 1$$

也就是对局部区域做一种模板匹配。

不同的滤波器可以提取不同特征，例如：

- 横向边缘
- 纵向边缘
- 某些纹理模式

7.3.2 填充（Padding）

卷积时，滤波器只能放在完整覆盖输入的位置上。这会导致一个问题：

经过卷积后，输出尺寸会变小。

例如：

- 输入大小 4×4
- 滤波器大小 3×3

那么输出只有 2×2。

如果网络层数很多，尺寸会不断缩小得很快。为了解决这个问题，可以在输入周围补零，这就叫做 **padding**。

例如给图像四周加一圈 0：

$$\text{原输入} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad \text{padding 后} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 \\ 0 & 3 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Padding 的作用是：

- 控制输出尺寸
- 让边缘像素也被充分利用
- 避免特征图缩小得太快

7.3.3 步幅（Stride）

滤波器在输入上滑动时，每次移动多少格，叫做 **stride**。
例如：

- stride = 1 表示每次移动 1 格
- stride = 2 表示每次移动 2 格

Stride 越大，输出特征图越小，因为采样得更稀疏。
因此卷积层的输出大小不仅和：

- 输入大小
- 滤波器大小
- padding

有关，也和 stride 有关。

7.3.4 输出大小的计算

如果输入大小为 $H \times W$，滤波器大小为 $FH \times FW$，padding 为 P，stride 为 S，那么输出大小可以写成：

$$OH = \frac{H + 2P - FH}{S} + 1$$

$$OW = \frac{W + 2P - FW}{S} + 1$$

其中：

- OH 表示输出高度
- OW 表示输出宽度

需要注意的是：

> 上式结果必须是整数，否则说明参数组合不合法。

这是卷积层设计时非常重要的一个公式。

7.4 三维数据的卷积

前面为了便于理解，只讨论了二维输入。但真正的图像通常不是二维，而是三维数据。
例如一张彩色图像通常具有：

- 高度（Height）
- 宽度（Width）
- 通道数（Channel）

例如 RGB 图像有 3 个通道。
因此，一个输入通常可以写成：

$$(C, H, W)$$

其中：

- C 表示通道数
- H 表示高度
- W 表示宽度

卷积时，滤波器也必须在通道维度上与输入匹配。
也就是说：

> 如果输入有 3 个通道，那么一个滤波器也必须覆盖这 3 个通道。

例如：

- 输入形状是 $3 \times 28 \times 28$
- 一个滤波器形状可以是 $3 \times 5 \times 5$

这个滤波器会同时看 3 个通道上的局部区域，然后输出一个数。

7.4.1 多个滤波器

实际卷积层通常不会只用一个滤波器，而是使用很多个滤波器。
为什么？
因为不同滤波器可以学习不同的特征：

- 有的滤波器擅长提边缘
- 有的滤波器擅长提纹理
- 有的滤波器擅长提某种局部形状

如果有 FN 个滤波器，那么输出就会有 FN 个通道。
因此，卷积层输出形状通常写成：

$$(FN, OH, OW)$$

这就是所谓的**特征图（feature map）**。

7.4.2 偏置

和全连接层一样，卷积层通常也会有偏置项。
只不过在卷积层中，偏置的加法方式是：

- 每个滤波器有一个偏置
- 该偏置会加到该滤波器输出的整张特征图上

所以如果有 FN 个滤波器，通常就有 FN 个偏置。

7.5 池化层

除了卷积层，CNN 中另一个非常重要的层是 **池化层（Pooling Layer）**。
池化层的核心作用是：

> 对局部区域进行“压缩”，保留主要信息，减少空间尺寸。

最常见的是 **最大池化（Max Pooling）**。
例如对一个 2×2 区域做最大池化，就是取其中最大的值。
示例：

$$\text{输入局部区域} = \begin{bmatrix} 1 & 2 \\ 5 & 3 \end{bmatrix} \qquad \text{最大池化输出} = 5$$

7.5.1 池化的作用

池化层常见的作用有：

- 降低特征图尺寸
- 减少计算量
- 提高对微小平移的鲁棒性
- 突出最显著的局部响应

池化可以理解为：

> 把局部区域中最重要的信息保留下来。

因此 CNN 经常采用这样的结构：

- 卷积
- 激活函数
- 池化

不断堆叠来提取越来越高级的特征。

7.5.2 池化层的特点

池化层和卷积层不一样，它通常没有需要学习的参数。
也就是说：

- 卷积层有权重和偏置

- 池化层没有参数

池化只是固定地做一种聚合操作，例如：

- 最大值
- 平均值

所以它的本质更像是一种特征压缩，而不是参数化变换。

7.6 CNN 的整体结构

卷积神经网络通常由以下部分组成：

1. 卷积层
2. 激活函数层
3. 池化层
4. 全连接层
5. 输出层

一个常见的结构是：

$$\text{卷积} \to \text{ReLU} \to \text{池化} \to \text{卷积} \to \text{ReLU} \to \text{池化} \to \text{全连接} \to \text{Softmax}$$

其中前面的卷积和池化部分主要负责：

- 提取局部特征
- 压缩空间结构
- 形成更抽象的表示

后面的全连接层主要负责：

- 基于这些高层特征进行分类或回归

所以可以把 CNN 分为两部分理解：

前半部分

负责“看图并提特征”。

后半部分

负责“根据特征做决策”。

7.7 卷积层的实现问题

如果直接用最朴素的方法实现卷积，会遇到一个问题：

需要大量重复的循环操作，效率较低。

例如：

- 遍历每个样本
- 遍历每个通道
- 遍历每个滤波器
- 遍历每个空间位置

这样实现虽然直观，但会比较慢。
所以需要更高效的实现方式。

7.8 im2col

为了提高卷积运算效率，一个很常见的技巧是：

im2col

它的思想是：

- 把输入中每个局部感受野都“展开”为一行
- 把卷积核也展开成列向量
- 然后把卷积运算转成一次大的矩阵乘法

这样就能充分利用 NumPy 对矩阵乘法的高效实现。
举例来说：

- 原始卷积是“滑窗 + 局部乘加”
- im2col 之后就变成“矩阵乘法”

虽然中间会引入一些额外内存开销，但速度通常更快。

7.8.1 im2col 的直观理解

假设输入是：

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

如果卷积核大小是 2×2，那么所有局部区域分别是：

- 左上 2×2
- 往右滑一格的 2×2
- 再往下滑一格的 2×2
- …

im2col 会把每个局部区域都拉成一行，例如：

$$\begin{bmatrix} 1 & 2 & 5 & 6 \\ 2 & 3 & 6 & 7 \\ 3 & 4 & 7 & 8 \\ 5 & 6 & 9 & 10 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

这样所有局部区域组成一个大矩阵。再把卷积核也展开后，就可以用 `np.dot()` 一次算完。

7.9 卷积层实现

基于 im2col，可以写出卷积层的前向传播结构。

实现思路大致如下：

1. 用 im2col 展开输入
2. 展开滤波器
3. 做矩阵乘法
4. 加偏置
5. reshape 回输出特征图

卷积层的代码结构通常会包含这些变量：

- W：滤波器参数
- b：偏置
- stride
- pad
- x：输入缓存
- col：展开后的输入
- col_W：展开后的滤波器

这为后面做反向传播也提供了基础。

7.10 池化层实现

池化层也可以借助 im2col 来高效实现。

以最大池化为例，思路是：

1. 把局部区域展开
2. 对每一行取最大值
3. reshape 回输出形状

最大池化在反向传播时需要额外记录：

- 前向传播时哪个位置是最大值

因为只有最大值对应的位置会接收到梯度，其他位置的梯度都是 0。
因此池化层实现通常会额外缓存一个 `arg_max`。

7.11 SimpleConvNet

掌握了卷积层和池化层之后，就可以实现一个简单的卷积网络。
一个典型结构可能是：

$$\text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Affine} \to \text{ReLU} \to \text{Affine} \to \text{Softmax}$$

这种网络虽然简单，但已经可以在 MNIST 上取得不错效果。
其参数通常包括：

- 卷积层权重和偏置
- 第一个全连接层权重和偏置
- 第二个全连接层权重和偏置

网络通常会实现这些方法：

- predict
- loss
- accuracy
- gradient

这和前面实现全连接网络时的结构是类似的，只是把前面的层换成了卷积和池化。

7.12 CNN 的特征

通过本章可以看到，CNN 相比普通全连接网络，最大的特点有三个：

7.12.1 局部连接

每个神经元只关注输入中的一个局部区域。

7.12.2 参数共享

同一个滤波器会在整个图像上重复使用。

7.12.3 保留空间结构

图像的二维结构不会被一开始就破坏掉。
这三个特点使 CNN 特别适合图像任务。

7.13 经典 CNN 简介

随着卷积神经网络的发展，出现了许多经典结构。

虽然本章主要是理解原理，但知道几个代表性网络也很重要：

7.13.1 LeNet

很早期的经典卷积网络，主要用于手写数字识别。

7.13.2 AlexNet

在 ImageNet 比赛中大幅推动 CNN 流行的重要模型。

7.13.3 VGG

结构非常规整，特点是大量使用 3×3 卷积。

这些模型虽然复杂程度不同，但都建立在本章讲的基本组件之上：

- 卷积层
- 激活函数
- 池化层
- 全连接层

因此，本章内容是理解现代视觉模型的重要基础。

7.14 本章小结

7.14.1 为什么需要 CNN

全连接层不适合直接处理图像，因为会破坏空间结构并带来大量参数。

7.14.2 卷积层的作用

卷积层通过局部连接和参数共享来提取局部特征。

7.14.3 卷积中的关键概念

需要掌握：

- 滤波器
- padding
- stride
- 通道
- 特征图

7.14.4 输出大小公式很重要

卷积层和池化层的输出尺寸必须能够正确计算。

7.14.5 池化层的作用

池化层负责压缩空间尺寸并突出主要特征。

7.14.6 im2col 是实现技巧

它把卷积运算转化为矩阵乘法，从而提高计算效率。

7.14.7 CNN 的整体结构

通常由：

- 卷积层
- 激活函数层
- 池化层
- 全连接层

组合而成。

7.14.8 CNN 的意义

CNN 通过保留空间结构和利用局部特征，大幅提升了图像任务的效果。

7.15 附：本章核心代码

下面给出一个更接近鱼书实现风格的代码版本，包含：

- `im2col`
- `col2im`
- 卷积层 `Convolution`
- 池化层 `Pooling`

```
import numpy as np


def im2col(input_data, filter_h, filter_w, stride=1, pad=0):
    N, C, H, W = input_data.shape
    out_h = (H + 2 * pad - filter_h) // stride + 1
    out_w = (W + 2 * pad - filter_w) // stride + 1

    img = np.pad(
        input_data,
        [(0, 0), (0, 0), (pad, pad), (pad, pad)],
        mode='constant'
```

```
    )

    col = np.zeros((N, C, filter_h, filter_w, out_h, out_w))

    for y in range(filter_h):
        y_max = y + stride * out_h
        for x in range(filter_w):
            x_max = x + stride * out_w
            col[:, :, y, x, :, :] = img[:, :, y:y_max:stride, x:x_max:stride]

    col = col.transpose(0, 4, 5, 1, 2, 3).reshape(N * out_h * out_w, -1)
    return col


def col2im(col, input_shape, filter_h, filter_w, stride=1, pad=0):
    N, C, H, W = input_shape
    out_h = (H + 2 * pad - filter_h) // stride + 1
    out_w = (W + 2 * pad - filter_w) // stride + 1

    col = col.reshape(N, out_h, out_w, C, filter_h, filter_w).transpose(0, 3, 4, 5, 1, 2)
    img = np.zeros((N, C, H + 2 * pad + stride - 1, W + 2 * pad + stride - 1))

    for y in range(filter_h):
        y_max = y + stride * out_h
        for x in range(filter_w):
            x_max = x + stride * out_w
            img[:, :, y:y_max:stride, x:x_max:stride] += col[:, :, y, x, :, :]

    return img[:, :, pad:H + pad, pad:W + pad]


class Convolution:
    def __init__(self, W, b, stride=1, pad=0):
        self.W = W
        self.b = b
        self.stride = stride
        self.pad = pad

        self.x = None
        self.col = None
        self.col_W = None

        self.dW = None
        self.db = None

    def forward(self, x):
        FN, C, FH, FW = self.W.shape
        N, C, H, W = x.shape
        out_h = (H + 2 * self.pad - FH) // self.stride + 1
        out_w = (W + 2 * self.pad - FW) // self.stride + 1

        col = im2col(x, FH, FW, self.stride, self.pad)
        col_W = self.W.reshape(FN, -1).T
```

```
        out = np.dot(col, col_W) + self.b
        out = out.reshape(N, out_h, out_w, -1).transpose(0, 3, 1, 2)

        self.x = x
        self.col = col
        self.col_W = col_W

        return out

    def backward(self, dout):
        FN, C, FH, FW = self.W.shape
        dout = dout.transpose(0, 2, 3, 1).reshape(-1, FN)

        self.db = np.sum(dout, axis=0)
        self.dW = np.dot(self.col.T, dout)
        self.dW = self.dW.transpose(1, 0).reshape(FN, C, FH, FW)

        dcol = np.dot(dout, self.col_W.T)
        dx = col2im(dcol, self.x.shape, FH, FW, self.stride, self.pad)

        return dx


class Pooling:
    def __init__(self, pool_h, pool_w, stride=1, pad=0):
        self.pool_h = pool_h
        self.pool_w = pool_w
        self.stride = stride
        self.pad = pad

        self.x = None
        self.arg_max = None

    def forward(self, x):
        N, C, H, W = x.shape
        out_h = (H - self.pool_h) // self.stride + 1
        out_w = (W - self.pool_w) // self.stride + 1

        col = im2col(x, self.pool_h, self.pool_w, self.stride, self.pad)
        col = col.reshape(-1, self.pool_h * self.pool_w)

        arg_max = np.argmax(col, axis=1)
        out = np.max(col, axis=1)
        out = out.reshape(N, out_h, out_w, C).transpose(0, 3, 1, 2)

        self.x = x
        self.arg_max = arg_max

        return out

    def backward(self, dout):
        dout = dout.transpose(0, 2, 3, 1)

        pool_size = self.pool_h * self.pool_w
```

```
        dmax = np.zeros((dout.size, pool_size))
        dmax[np.arange(self.arg_max.size), self.arg_max.flatten()] = dout.flatten()

        dcol = dmax.reshape(-1, pool_size)
        dx = col2im(dcol, self.x.shape, self.pool_h, self.pool_w, self.stride, self.pad)

        return dx
```